

Executive Summary

This document provides an exhaustive, step-by-step breakdown of a comprehensive Spring Core tutorial focusing on Circular Dependencies, Bean Scopes, and Bean Initialization (Lazy vs. Eager). The core takeaway is that while Spring offers various mechanisms (like Field Injection or the `@Lazy` annotation) to resolve circular dependencies, these are fundamentally architectural flaws that violate the Single Responsibility Principle. A robust, enterprise-grade application should rely on proper code design rather than framework hacks. Furthermore, understanding how the Inversion of Control (IoC) container handles bean lifecycles—specifically when beans are created (Eager/Lazy) and how many instances are created (Singleton/Prototype)—is crucial for writing optimized Spring applications.

Chronological Deep-Dive

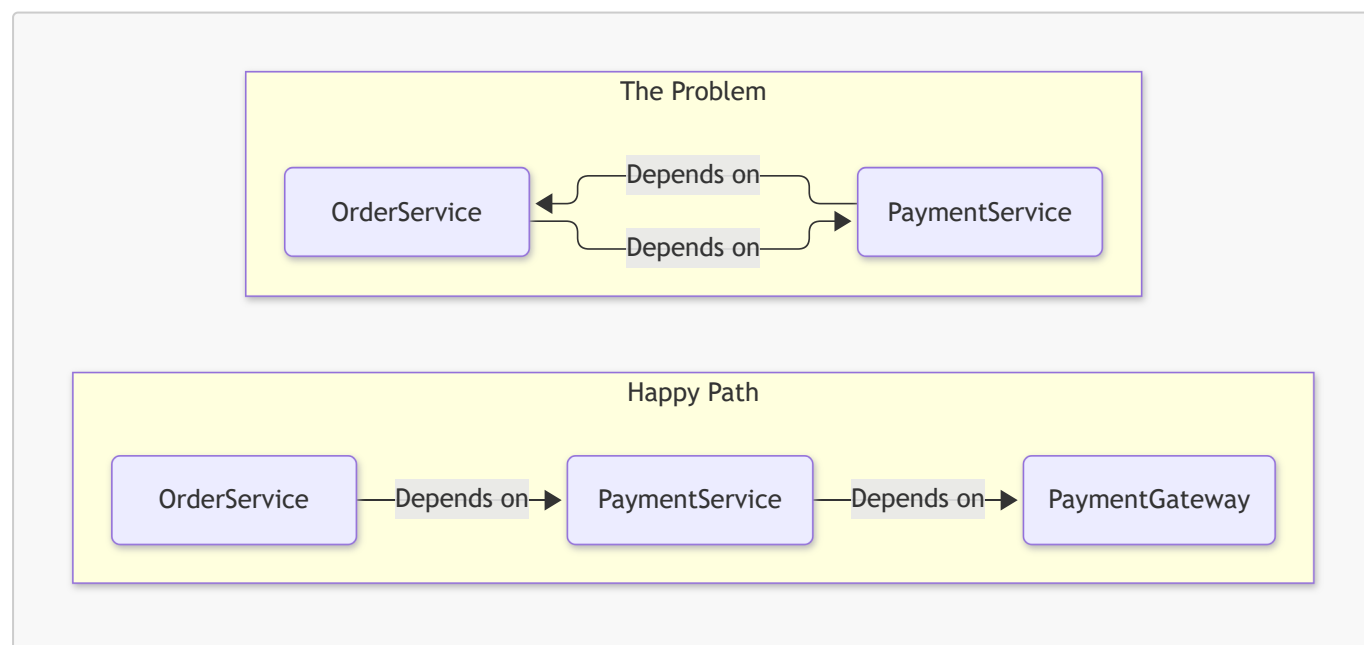
Introduction to Circular Dependency

In the Spring Framework, Dependency Injection (DI) allows the IoC (Inversion of Control) container to manage class dependencies. A standard dependency flow is linear. For example, if an `OrderService` requires a `PaymentService` to function, the `OrderService` is dependent on the `PaymentService`.

If we extend this, `PaymentService` might depend on a `PaymentGateway`. Spring handles this effortlessly:

1. It creates the `PaymentGateway`.
2. It injects the `PaymentGateway` into a new `PaymentService`.
3. It injects the `PaymentService` into a new `OrderService`.

The Circular Problem: A Circular Dependency arises when two or more classes depend on each other directly or indirectly. For instance, if `Class A` depends on `Class B`, and `Class B` depends on `Class A`, the IoC container faces a chicken-and-egg problem. It cannot determine the starting point for bean creation.



Setting up the Spring Environment

To demonstrate this practically, a new Maven project is established with the following setup:

- **Dependencies:** The `spring-context` dependency (version 7.0.7 for Spring Boot 4 compatibility) is added to the `pom.xml`. This brings in the core Spring IoC features.
- **Application Context:** The `AnnotationConfigApplicationContext` is used to boot up the IoC container in the `main` method.
- **Configuration Class:** An `AppConfig` class is created, annotated with `@Configuration` and `@ComponentScan`.

A Hidden Detail About `@Configuration`: The `@Configuration` annotation is actually a meta-annotation that internally uses `@Component`. This means the configuration class itself is instantiated and managed as a bean by the Spring IoC container. This is necessary because Spring needs an instance of the configuration class to invoke the `@Bean` methods defined inside it.

The Constructor Injection Trap

To simulate the issue, two classes are created and annotated with `@Component`:

1. `OrderService`
2. `PaymentService`

Both classes use **Constructor Injection** to request their dependencies.

- `OrderService`'s constructor requires a `PaymentService` object.
- `PaymentService`'s constructor requires an `OrderService` object.

When a method like `placeOrder()` is called, it delegates to `PaymentService.pay()`. To simulate entangled responsibilities, `pay()` then calls back to `OrderService.getOrderDetails()`.

When the Spring application runs, it immediately crashes. The IoC container attempts to resolve the graph but realizes neither object can be instantiated first.

The Exception: Spring throws a `BeanCurrentlyInCreationException`.

- **Why?** Spring starts creating `OrderService`. It hits the constructor and realizes it needs `PaymentService`. It pauses `OrderService` (putting it in a "currently in creation" state) and starts creating `PaymentService`. `PaymentService`'s constructor asks for `OrderService`. Spring looks at `OrderService`, sees it is already in the process of being created (not finished), and panics, throwing the exception and explicitly warning the developer about an "unresolvable circular reference."

Replicating Circular Dependency in Plain Java

To prove that Circular Dependency is a fundamental architectural flaw and not a "Spring problem," the video steps away from Spring annotations and recreates the scenario using raw Java classes (`Class A` and `Class B`).

If you attempt to inject dependencies via constructors in plain Java:

```
// Conceptual representation of the plain Java failure
A a = new A(new B(new A(new B(...))));
```

To try and hack around this, one might create default constructors that instantiate the other class:

- A's constructor calls `new B()`
- B's constructor calls `new A()`

When you execute `new A()`, the JVM enters an infinite loop of constructor calls. A calls B, B calls A, A calls B, and so on.

The Result: A `StackOverflowError`. The memory stack fills up with endless constructor execution frames until the application violently crashes.

Environment	Error Encountered	Why it happens
Spring IoC	<code>BeanCurrentlyInCreationException</code>	Spring detects the cyclic dependency graph during its bean resolution phase and gracefully aborts.
Plain Java	<code>StackOverflowError</code>	The JVM continuously allocates stack frames for recursive constructor calls until memory is exhausted.

Resolving Circular Dependency with Field & Setter Injection

If constructor injection completely blocks the IoC container, how does Spring offer a workaround? The answer lies in switching to **Field Injection** (using `@Autowired` directly on the class properties) or **Setter Injection**.

When you use field or setter injection, you fundamentally change how the objects are instantiated. You separate the **creation** of the object from the **injection** of its dependencies.

Because a constructor with parameters isn't strictly required to create the initial object footprint, Spring can call the default, no-argument constructor to create "empty" or partial objects first, and then wire them together afterward.

A Quick Tangent: How Field Injection Works via Java Reflection

You might wonder: *If a field is marked `private` (like `private PaymentService paymentService;`), and there are no setter methods, how does Spring inject the object into it?*

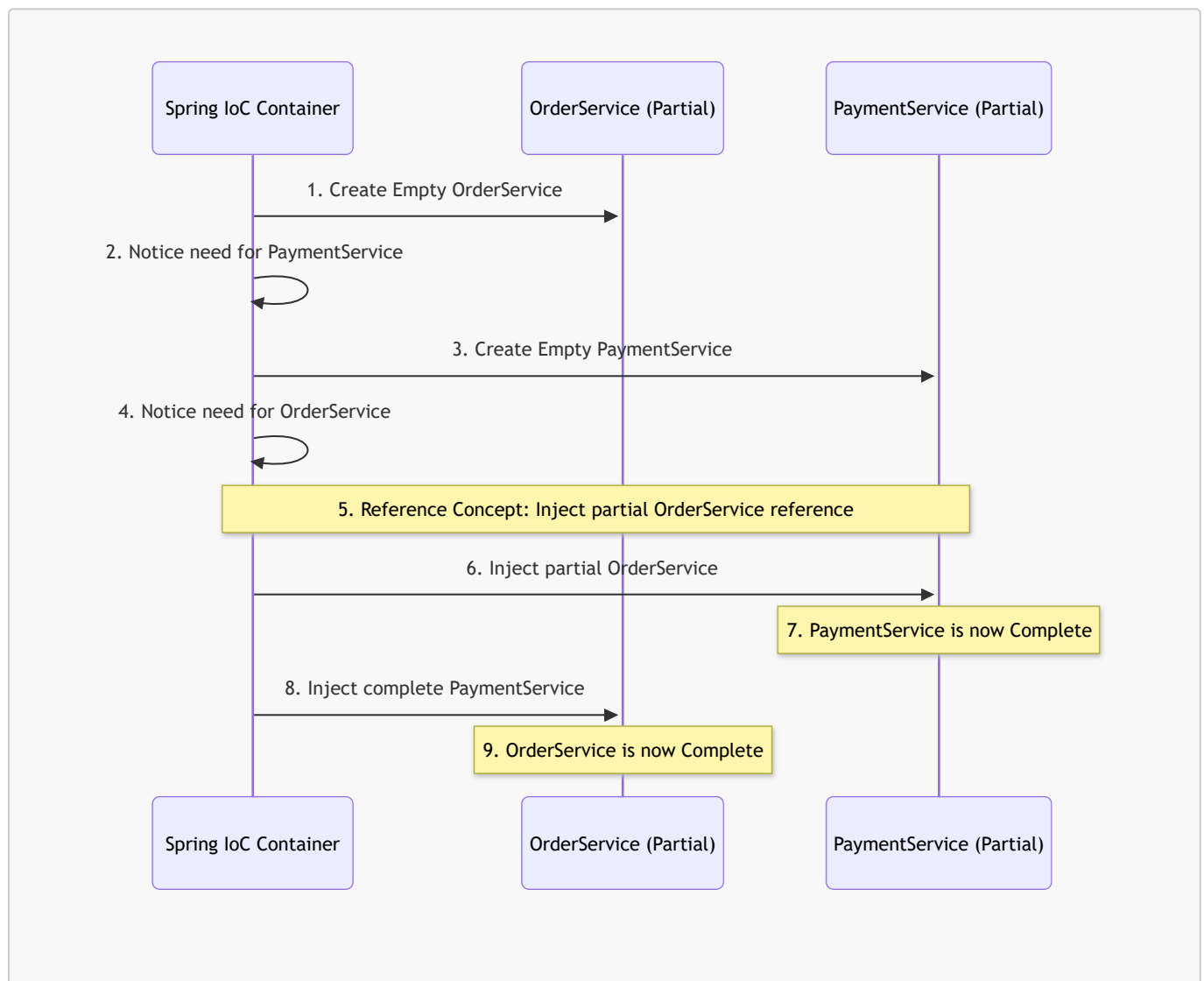
The Spring IoC container uses **Java's Reflection API**. When Spring parses the bean definitions, it accesses the class metadata. Reflection allows Spring to bypass standard Java access modifiers, directly reaching into the private members of the class to inject the dependency. While powerful, this is part of the "magic" of Spring that operates under the hood.

The Reference Concept: Under the Hood of the IoC Container

Let's look at exactly how the IoC container navigates a circular dependency when field injection is used. Spring employs a mechanism we can call the **Reference Concept**.

Here is the exact step-by-step flow Spring takes to resolve the circular lock between `OrderService` and `PaymentService`:

1. **Start Creation:** Spring attempts to create `OrderService`.
2. **Create Partial Object:** It invokes the default constructor to create a raw, partial `OrderService` object (dependencies are currently null).
3. **Detect Dependency:** Spring realizes `OrderService` needs `PaymentService`.
4. **Create Dependent Partial Object:** Spring pauses the injection phase for `OrderService` and invokes the default constructor of `PaymentService` to create a partial `PaymentService` object.
5. **Detect Circular Dependency:** Spring realizes `PaymentService` needs `OrderService`.
6. **Inject Partial Reference:** Instead of panicking, Spring checks its cache and says, "I have a partially created `OrderService`!" It injects the memory reference of this *partial* `OrderService` into the `PaymentService`.
7. **Complete Dependent Object:** `PaymentService` is now fully assembled.
8. **Complete Initial Object:** Spring returns to `OrderService` and injects the fully assembled `PaymentService` into it.



The Reality Check: Why You Should Avoid This

Even though Spring *can* resolve circular dependencies using field or setter injection, **it is considered a bad coding practice**. In a professional, enterprise-level application, circular dependencies should not exist.

Why is it a Bad Practice?

If `OrderService` and `PaymentService` are so deeply entangled that they require each other to exist, you have violated the **Single Responsibility Principle (SRP)**—the 'S' in SOLID design principles. Their responsibilities are not properly separated, resulting in tightly coupled code that is difficult to maintain, scale, and test.

Spring Boot's Stance

Spring Boot actively discourages this. Starting from Spring Boot 2.6, circular dependencies are **banned by default**. If you run the exact same field-injection code in a modern Spring Boot application, it will fail.

You *can* force Spring Boot to allow it by adding a specific property to your `application.properties` file: `spring.main.allow-circular-references=true`. However, the default value is `false` for a reason.

The Real Solution: Code Refactoring

Instead of relying on framework hacks, you must refactor the code. In our scenario, the circularity happens because `PaymentService.pay()` finishes its job and then calls back to `OrderService.getOrderDetails()`.

The Fix: `PaymentService` should only handle payments. It shouldn't care about order details. We must strip that responsibility away from it. `OrderService` should call `pay()`, wait for it to finish, and then sequentially call `getOrderDetails()` internally. Once refactored, the dependency becomes beautifully linear, and the circular lock disappears.

Bean Scopes: Singleton vs. Prototype

By default, how many instances of a bean does Spring create? This brings us to **Bean Scopes**. The two primary scopes in core Spring are Singleton and Prototype.

The Singleton Scope (The Default)

When a bean is configured as a Singleton (which is the default behavior if you specify nothing), the Spring IoC container creates **exactly one instance** of that object per Bean Definition.

If you use `context.getBean(OrderService.class)` twice and store the results in `order1` and `order2`, doing an identity check (`order1 == order2`) will return `true`. They point to the exact same memory location. Multiple classes requesting the dependency will all be handed a reference to this single shared object.

A Crucial Nuance: Spring Singleton vs. Java Singleton: In standard low-level system design, the Gang of Four (GoF) Singleton Pattern dictates that only one object of a class can *ever* exist in the JVM. Spring's Singleton is different. It means one object *per bean definition*. If you write two separate `@Bean` methods in your `@Configuration` class that both return `new OrderService()`, Spring will create and manage *two* distinct Singleton objects inside the container. Furthermore, nothing stops you from manually writing `new OrderService()` in your code to create a third, unmanaged instance.

The Prototype Scope

If you annotate your class with `@Scope("prototype")`, the behavior flips. Every single time you request the bean—whether by calling `getBean()` or through dependency injection in multiple classes—Spring will create

and hand you a **brand new instance** of that object.

If **Class A** and **Class B** both inject a Prototype **OrderService**, two distinct **OrderService** objects are instantiated.

When to use which?

Feature	Singleton Scope	Prototype Scope
Object Count	One instance per Bean Definition.	A new instance per request/injection.
Best Candidate	Stateless Classes. Services, Managers, Repositories.	Stateful Classes. Entities, DTOs, User objects.
Why?	A Service has methods but holds no user-specific data. Sharing it saves memory.	A User object holds specific data (Name, Age). Sharing it would overwrite user states.

(Note: There are other scopes like **request**, **session**, and **application**, but those only apply when building web applications with Spring Web MVC.)

Bean Initialization: Eager vs. Lazy

Initialization dictates *when* the bean is created by the IoC container.

Eager Initialization

By default, beans are **Eagerly Initialized**. The moment you start the Spring application and the Application Context boots up, Spring scans, defines, and creates all the Singleton beans immediately, even if no other code has requested them yet.

Why Eager by Default? The "Fail-Fast" Principle: Spring wants your application to "fail fast." If you have a wiring issue, a missing dependency, or a typo in a bean name, Eager Initialization guarantees that the application crashes the moment you try to start it. If beans were delayed, your app might boot successfully, only to crash hours later in production when a user triggers a specific piece of code that requests a misconfigured bean.

Lazy Initialization

If a bean is very resource-intensive to create and you only want it instantiated when it is actually needed, you can use the **@Lazy** annotation.

If you mark **@Lazy** on an **OrderService**, starting the application will result in an empty container. The object is only created the precise moment **context.getBean()** is called, or when another bean requests it as a dependency.

Scope and Initialization Rules:

- **Singleton Beans:** Eager by default, but can be explicitly made Lazy using **@Lazy**.

- **Prototype Beans: Always Lazy by default.** Because a prototype bean promises a *new* object every time it's requested, it makes zero logical sense for the container to proactively create one on startup.

You can globally make your entire application Lazy by setting `spring.main.lazy-initialization=true` in your properties file. If you do this, you can selectively make specific beans Eager by annotating them with `@Lazy(false)`.

The Ultimate Hack: Using @Lazy to Fix Circular Constructor Dependencies

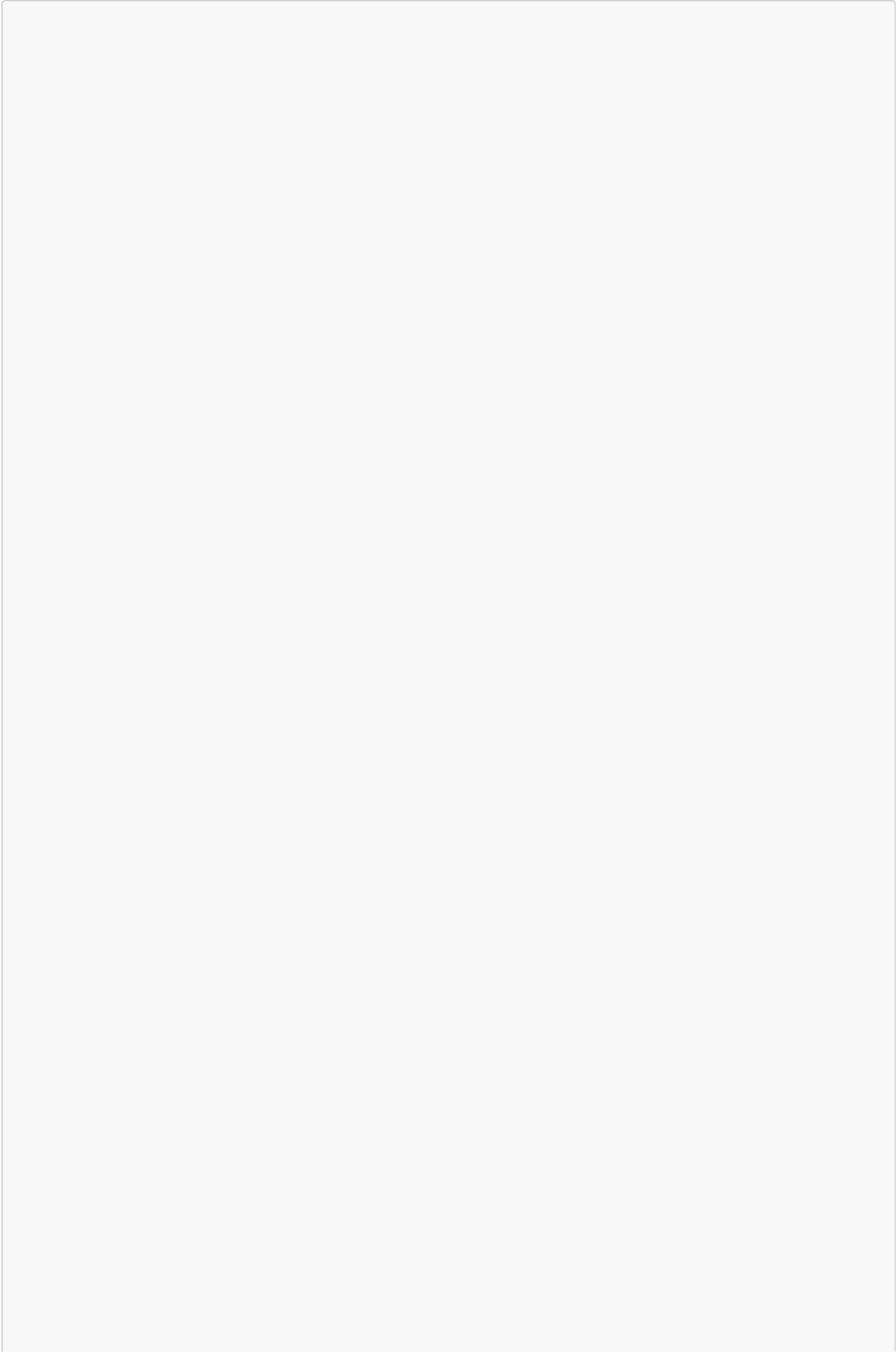
Earlier, we established that Circular Dependencies utilizing Constructors will crash the application because Spring can't create partial objects via constructors. However, the `@Lazy` annotation provides a powerful trick to bypass this using the **Proxy Pattern**.

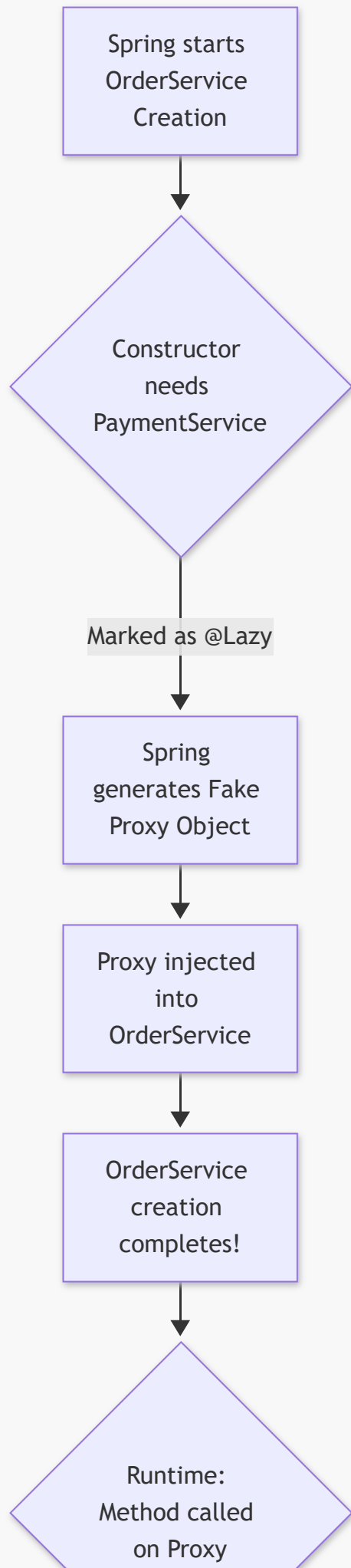
If `OrderService` requires `PaymentService` in its constructor, and `PaymentService` requires `OrderService` in its constructor, you can place the `@Lazy` annotation directly inside the constructor parameter list of `OrderService`:

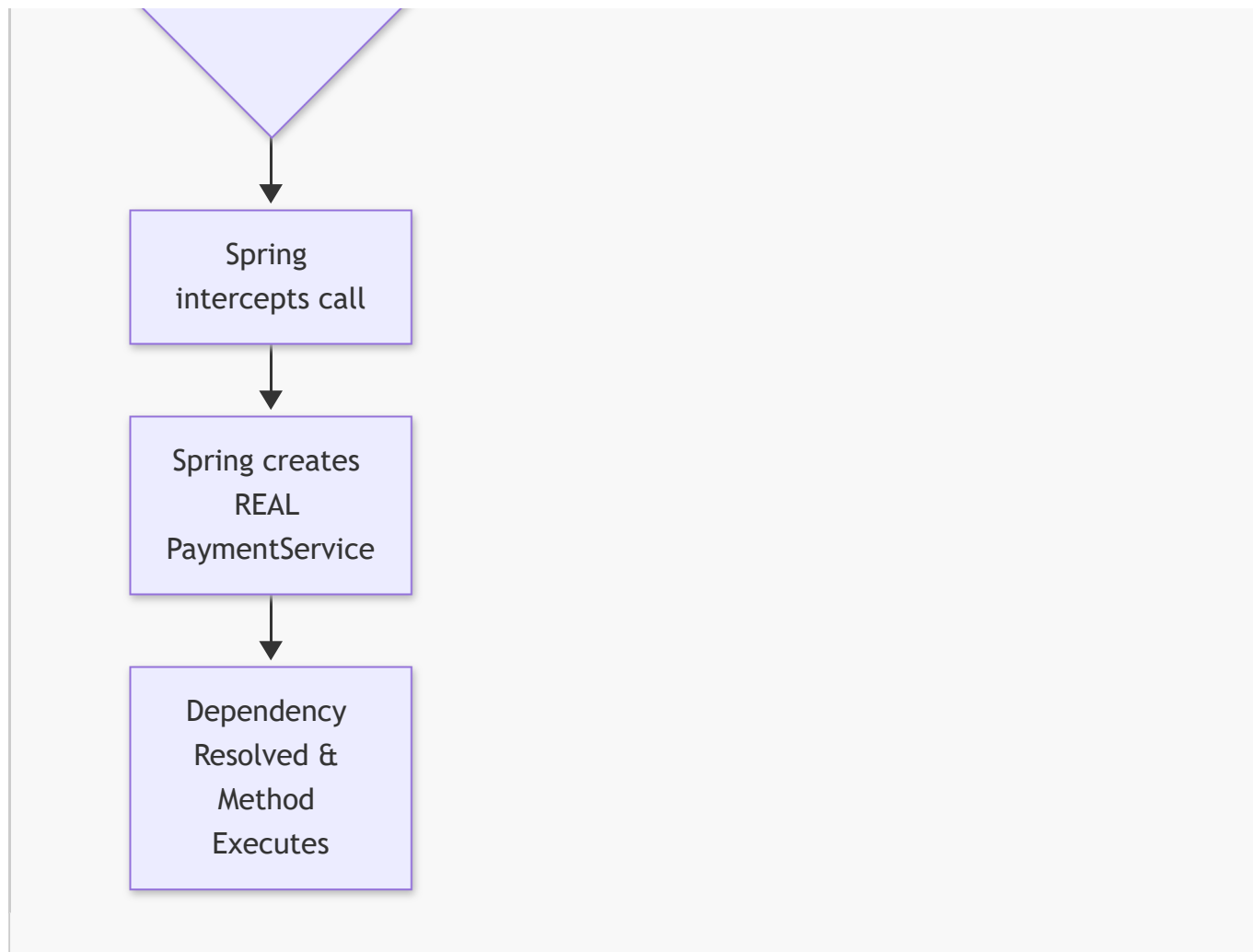
```
@Autowired
public OrderService(@Lazy PaymentService paymentService) {
    this.paymentService = paymentService;
}
```

How the Proxy works here:

1. Spring starts creating `OrderService`.
2. It hits the constructor and sees it needs `PaymentService`, but it's marked `@Lazy`.
3. Spring says, "I won't create the real `PaymentService` yet. Instead, I'll generate a **Proxy Object**." This proxy is a fake, hollow object that masquerades as a `PaymentService`.
4. Spring injects the Proxy into `OrderService`.
5. `OrderService` successfully finishes creation!
6. Later, when `OrderService` actually calls a method on the proxy (like `paymentService.pay()`), Spring intercepts the call, instantly instantiates the *real* `PaymentService`, swaps out the proxy, and executes the method.







While this is an elegant technical solution provided by the framework, the core lesson remains: use these tools to understand the framework's depth, but strive to write cleanly separated, non-circular code architecture first.

Actionable Takeaways

- **Avoid Circular Dependencies:** Treat circular references as a design flaw (a violation of the Single Responsibility Principle) rather than a framework puzzle to solve. Always prefer refactoring your logic to establish a linear dependency flow.
- **Constructor over Field Injection:** Continue using Constructor Injection as your primary method for dependency wiring. It enforces immutability and immediately highlights circular design flaws (crashing on startup) rather than silently patching them like Field Injection does.
- **Align Scope with State:** Use the default **Singleton** scope for stateless components (Services, Controllers, Repositories). Explicitly use **Prototype** scope for stateful components (like data-holding objects or DTOs) to prevent data corruption across threads.
- **Understand Spring's Singleton Nuance:** Remember that Spring's Singleton means "one instance per bean definition," not "one instance per JVM." You can have multiple singletons of the same class if you define them separately in configuration classes.
- **Embrace Fail-Fast:** Leave the default Eager Initialization active for your Singletons. Finding wiring errors instantly on startup is far better than encountering them in a live production environment.
- **Strategic Laziness:** Only use the **@Lazy** annotation for exceptionally heavy, resource-intensive beans that are rarely used, or as an absolute last-resort band-aid for legacy code entangled in circular

constructor dependencies.